

Title: Constructing a Pokémon World: Analyzing Language Across Types and Generations

Group Name: Lanturns

Members: Caroline Clippinger (Leader), Ethan Wang, Thieron Cook

Class/Section: DS 4002-002 9/10/2026

Goal: We will analyze Pokémon descriptions to identify patterns in descriptive vocabulary across Pokémon types and generations to determine which types and generations cluster together and which words characterize these clusters; success will be contingent on the production of well-separated clusters (high silhouette scores) with semantically cohesive and interpretable word patterns (distinct relative word frequencies within versus outside each cluster).

Research Question: How do the descriptive characteristics of Pokémon differ across types and generations, and which types or generations exhibit similar descriptive language?

Model Approach:

1. Extract Pokédex entry data using the PokeAPI and using a combination of tidyverse, tidytext, and udpipe packages in R to tokenize the words within entries using a Natural Language Processing (NLP) model and rank resulting lemmas based on their TF-IDF scores across Pokémon types and generations.
2. Create matrices where the columns are the 150 words with the highest averaged TF-IDF score between the Pokémon types and generations analyses, and the rows are each unique Pokémon type or generation.
3. Conduct a hierarchical clustering analysis on both matrices and determine the best cluster configuration using a combination of averaged silhouette scores and semantic cohesiveness of the characteristic word profiles.

Executive summary:

This document establishes the Pokédex entry dataset used to compare descriptive language across Pokémon types and generations; it covers all 1,025 Pokémon species and is sourced from PokéAPI under a BSD 3-Clause license. Exploratory analysis identified one structural issue: missing typing for 37 form-varying Pokémon, which is addressed in the analysis plan below. The step-by-step Hierarchical Clustering Analysis plan is defined alongside necessary parameters and evaluation metrics.

Data set establishment details

<https://github.com/PokeAPI/pokeapi/blob/master/LICENSE.md>

Data summary: The dataset contains 1,025 rows, one per Pokémon species, covering Pokédex IDs 1 to 1025 (Generations I through IX) with eight columns. The central feature is `pokedex_entry`, an English text description for each Pokémon, averaging 20.8 words (median 20, range 10 to 41) across roughly 21,900 total words and 4,241 unique word forms. Supporting features include: introduction generation, primary and secondary elemental type, species classification (genus), and assigned body color. Because our analysis aggregates text to the type and generation level, two counts are structurally important: the data covers all 18 elemental types and all 9 generations, and of the 988 Pokémon with complete typing, 486 are single-typed, and 502 carry a secondary type. The data comes from PokéAPI [1], a free, open API that

serves canonical Pokémon game data, where the descriptive text is returned in the `flavor_text_entries` field of the `pokemon-species` endpoint. The CSV used for this analysis is stored in our project GitHub repository at `/project1/pokedex.csv`.

Provenance: The Pokédex entries come from in-game text written by GAME FREAK and published by Nintendo/The Pokémon Company across nine generations of Pokémon games (1996 to present). The PokéAPI project, an all-volunteer open-source effort, transcribed and structured that text into a database and serves it through a free public API. Our group retrieved the data on September 10, 2026 using a Python script, preserved in `project1/Analysis.ipynb`, which requested all 1,025 species from the `pokemon-species` endpoint, pulled elemental typing from the `pokemon` endpoint, selected the first English-language flavor text and genus for each species, and wrote the results to `project1/pokedex.csv` using the `pandas` package.

License: PokéAPI is released under the BSD 3-Clause license [2], which permits use, modification, and redistribution of its data. The license requires that we reproduce the copyright notice and disclaimer in our repository, and prohibits using the PokéAPI name or the names of its contributors to endorse or promote our work without written permission.

Data dictionary

Feature	Type	Description	Values / Range	Missing	Uncertainty
<code>id</code>	Integer	Pokédex number; unique row identifier	1 to 1025	0	None. No duplicates.
<code>name</code>	String	Species name, lowercase	1,025 unique	0	None.
<code>genus</code>	String	Species classification shown on the Pokédex screen (e.g. Seed Pokémon)	715 unique	0	Not unique. Evolutionary families often share a genus, so it is not an identifier.
<code>type_1</code>	String	Primary elemental type (slot 1)	18 categories	37	Blank for species with form-dependent typing. Slot order is a game convention rather than a ranking, so <code>type_1</code> is not more the Pokémon's type than <code>type_2</code> .
<code>type_2</code>	String	Secondary elemental type (slot 2)	18 categories	523	Of the 523 blanks, 486 are true single-type Pokémon and 37 are the unknown typing cases above. Blank is therefore meaningful for most rows and must not be imputed.
<code>color</code>	String	PokéAPI's assigned primary body color	10 categories; blue most common (170)	0	Assigned by PokéAPI rather than by the games. A single subjective label for multicolored designs.

generation	String	Game generation in which the species was introduced	generation-i to generation-ix; 72 to 156 per group	0	Reflects species debut, not necessarily the game the flavor text came from. See unknowns.
pokedex_entry	String	English text description of the species	10 to 41 words, mean 20.8	0	Source game version is not recorded in this file. See unknowns.

One Question Answered

Does it matter whether dual-type Pokémon are counted once or twice? Figure 1 shows the number of Pokémon in each type, split by whether that type appears in the primary or the secondary slot. Assigning each Pokémon only to its primary type produces severely unbalanced groups. Flying is down to 8 Pokémon despite appearing on 102 in total, because it is overwhelmingly a secondary type. Additionally, Poison, Fairy, Steel, and Ground types lose roughly half their members for the same reason. Several types would be left with too little text to produce stable TF-IDF values. Counting each Pokémon toward both of its types instead yields group sizes ranging from 47 (Ice) to 146 (Water), with text volumes of roughly 1,000 to 3,100 words per type. This is still imbalanced, but every type has enough material to characterize, which is why our feature engineering pivots the primary and secondary type columns so that all 502 dual-type Pokémon are represented in both of their type groups. Figure 1 also reflects a second finding: it counts only the 988 Pokémon with recorded typing, because 37 form-varying species such as Deoxys, Aegislash, and Urshifu were left blank by our collection script. PokéAPI stores typing at the form level rather than the species level, so a request using the bare species name returns no result for these Pokémon. Those values will be researched and appended during preprocessing.

One Current Unknown

We do not know which game version each Pokédex entry was drawn from. The CSV records one English entry per Pokémon, but not its source game. Because the collection script selects the first English flavor_text_entry returned by the API, and PokéAPI orders those entries by game version, each Pokémon is represented by the description from its debut game. Generation and source game are therefore confounded. As Figure 2 shows, Generation III entries average 30.5 words against a 16- to 23-word baseline for every other generation, reflecting the longer Ruby and Sapphire entry format. This is a threat to the generation clustering specifically, since apparent generational language patterns may partly reflect the writing conventions of a single game. Two features of our analysis plan mitigate it. First, our features are TF-IDF values rather than raw counts, and our distance metric is cosine distance, both of which are insensitive to differences in entry length. Residual confounding will be acknowledged as a limitation when we present results.

Exploratory Plots

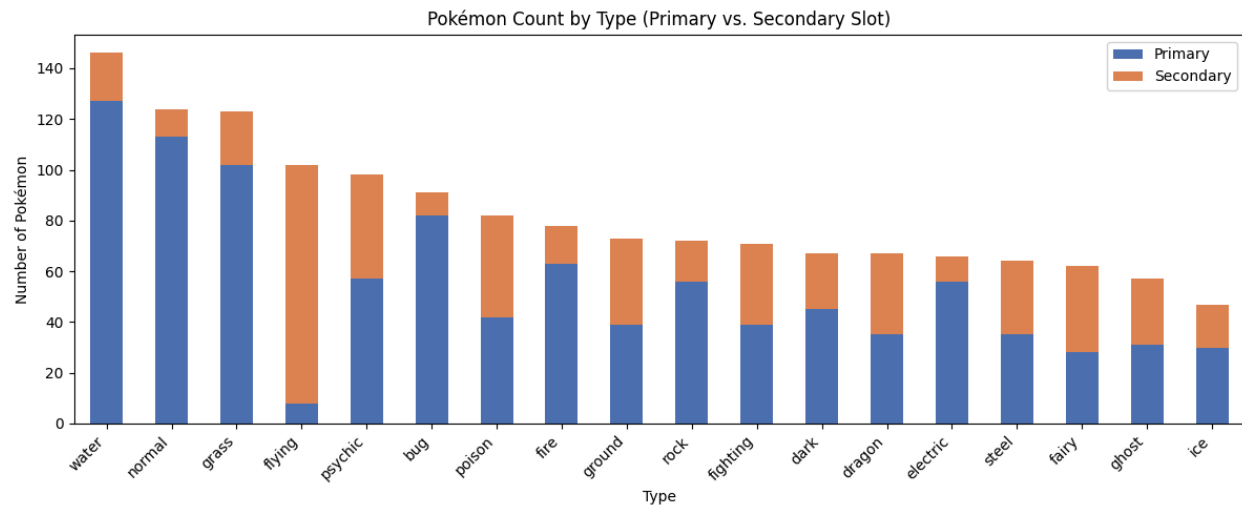


Figure 1. Number of Pokémon per elemental type, split by whether the type appears in the primary or secondary slot. Counting both slots, group sizes range from 47 (Ice) to 146 (Water). Flying is the extreme case, with 8 primary and 94 secondary assignments. Counts exclude the 37 Pokémon with no recorded typing.

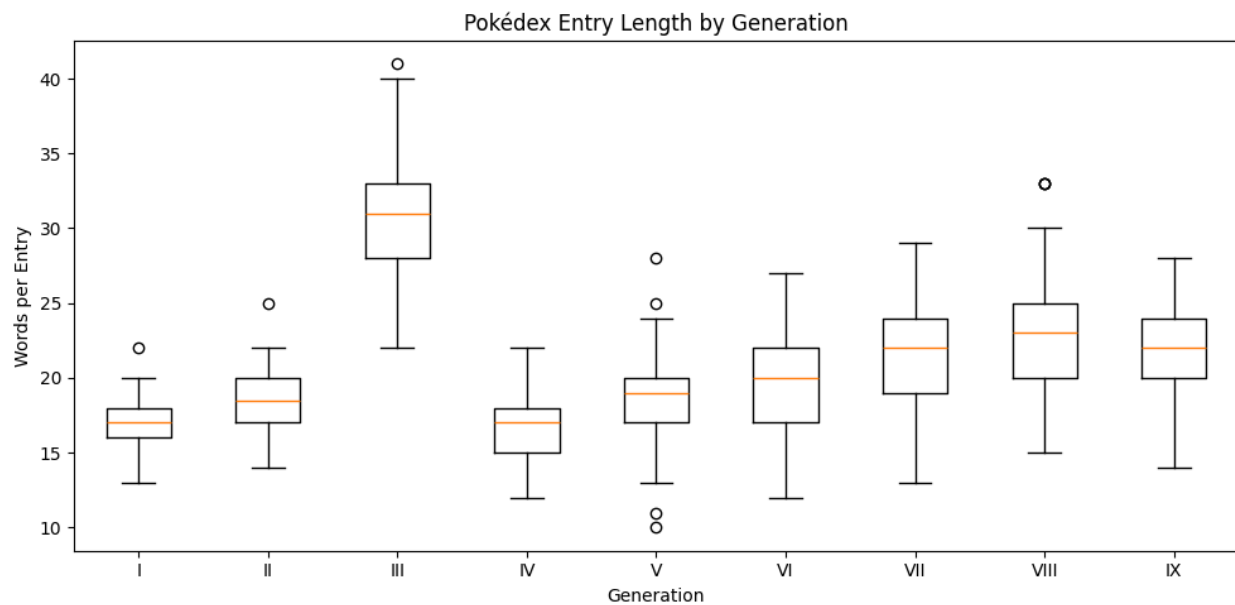
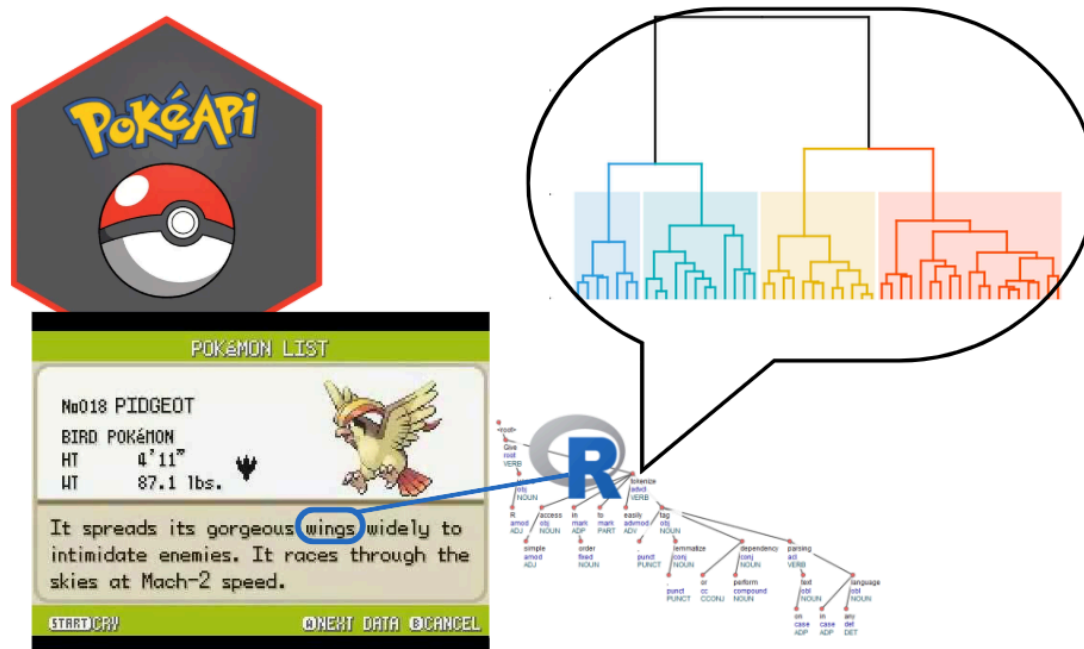


Figure 2. Distribution of Pokédex entry length in words, by generation. Generation III is a clear outlier, with a median near 31 words against a 16 to 23 word median elsewhere, reflecting the longer entry format of the Ruby and Sapphire games from which those entries are drawn.

Analysis Plan



Paragraphs:

Data Acquisition: We will obtain our data from the PokeAPI to get the variables needed for analysis, including Pokémon name, type(s), generation, and Pokédex entry. Using the pandas package, we created a data frame and exported it as a CSV file to be cleaned in R.

Preprocessing: We will check the CSV file for any missing data in the name, type, generation, and Pokédex entry columns. Any missing values that are not name values will be researched and appended to our dataset so that we have the most complete information. To prepare the textual data for analysis, we will process the Pokédex entries using Natural Language Processing (NLP) techniques available in the `udpipe` [3] R package. Specifically, we will tokenize the entries to extract word tokens, which we will lemmatize using the package's pre-trained models to reduce redundant words and ensure grammatical variations are consistently addressed. Afterwards, we will filter out words shorter than two characters and identifiable filler words using the "stop words" data file stored in the `tidytext` [4] R package. Finally, any words that are Pokémon names, words that identify a generation (specifically region names, e.g., Unova or Sinnoh), words that are equivalent to Pokémon types (e.g., dragon, ground), and words that are considered Pokémon-game language (e.g., pokémon, region, battle, etc.) will also be removed.

Feature engineering: The features of our parallel hierarchical clustering models for Pokémon type and generation will consist of the 150 words (or lemmas) with the highest averaged TF-IDF score between type and generation. Term frequency-inverse document frequency (TF-IDF) is a technique that weights words according to their frequency within a specified document (TF) while reducing the influence of words that occur across multiple documents (IDF) [SOURCE]. Thus, a higher TF-IDF score indicates a word is more informative. Using the `tidytext` [4] R package, we will calculate TF-IDF for each distinct word within each type or generation. Before doing this, we will use the `tidyverse` [5] R package to pivot-longer our Pokémon data on the primary and secondary type columns so that dual-type Pokémon will be considered in the analysis of both unique types. Afterward, we will take the average TF-IDF score for each word across types and generations separately. Finally, we will take the average of these averaged

type and generation TF-IDF scores for the unique words. Our features, or unique descriptive words, will be selected by selecting words with the highest averaged type-generation TF-IDF score.

Preparing and completing HCA: The selected 150 descriptive words will be used to build two matrices, where each row represents a unique type or generation and each column a word. The corresponding TF-IDF value for each word within each type or generation will populate the matrices. We will complete this by filtering the respective TF-IDF datasets to those 150 words and pivoting wider using the tidyverse [5] R package. Once generated, these matrices will be saved as CSV files to be imported into Python for completing the HCA using the SciPy [6] package.

Methodology and evaluation: For our HCA, we will use cosine distance to measure the similarity between Pokémon types and generations. Cosine distance is well-suited for comparing “high-dimensional” TF-IDF “text vectors,” making it commonly used for representing documents [7]. Additionally, we will use average linkage, “a balanced general-purpose choice”, to determine distances between clusters, instead of the more frequently used Ward’s method, which is incompatible with cosine distance [8]. To determine the best number of clusters, we will generate dendrograms to visualize the hierarchical relationships among Pokémon types and generations, indicating when the largest distance between merges occurred (represented by tall vertical lines). Finally, we will evaluate the different solutions by comparing average silhouette scores for each tested number of clusters to evaluate cluster separation [9]. Additionally, we will assess semantic interpretability by extracting what words are most characteristic of each cluster by computing relative word frequencies within versus outside clusters.

Quantifiable Goal: Identification of the ideal number of clusters (k) for grouping Pokémon based on type and generation as determined by the configuration with the greatest average silhouette score and most semantically cohesive characteristic-word profile.

Work Assignments

Member	Responsibility
Ethan Wang	Dataset establishment and details; Organizing GitHub
Caroline Clippinger	Data preprocessing and cleaning; Establishing analysis plan
Thieron Cook	Conducting analysis; Obtaining the dataset

References

- [1] PokéAPI, "PokéAPI: The RESTful Pokémon API." [Online]. Available: <https://pokeapi.co>. [Accessed: Sep. 17, 2026].
- [2] PokéAPI Contributors, "LICENSE.md," PokeAPI/pokeapi GitHub repository. [Online]. Available: <https://github.com/PokeAPI/pokeapi/blob/master/LICENSE.md>. [Accessed: Sep. 17, 2026].
- [3] J. Wijnffels, BNOSAC, M. Straka, and J. Straková, "udpipe: Tokenization, Parts of Speech Tagging, Lemmatization and Dependency Parsing with the 'UDPipe' 'NLP' Toolkit," *CRAN*, ver. 0.8.16, 2026. [Online]. Available: <https://cran.r-project.org/web/packages/udpipe/index.html>. [Accessed: Sep. 16, 2026].
- [4] J. Silge, D. Robinson, G. De Queiroz, C. Fay, E. Hvitfeldt, O. Keyes, K. Misra, T. Mastny, and J. Erickson, "tidytext: Text Mining using 'dplyr', 'ggplot2', and Other Tidy Tools," *CRAN*, ver. 0.4.3, 2025. [Online]. Available: <https://cran.r-project.org/web/packages/tidytext/index.html>. [Accessed: Sep. 16, 2026].
- [5] H. Wickham and RStudio, "tidyverse: Easily Install and Load the 'Tidyverse'," *CRAN*, ver. 2.0.0, 2023. [Online]. Available: <https://cran.r-project.org/web/packages/tidyverse/index.html>. [Accessed: Sep. 16, 2026].
- [6] SciPy Developers, "Hierarchical clustering (scipy.cluster.hierarchy)," SciPy Documentation, SciPy v1.18.0. [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/cluster.hierarchy.html>. [Accessed: Sep. 16, 2026].
- [7] MetricGate, "Distance vs Similarity Metrics for Clustering," *MetricGate*. [Online]. Available: <https://metricgate.com/blogs/distance-vs-similarity-metrics-clustering/>. [Accessed: Sep. 16, 2026].
- [8] MetricGate, "Choosing a Linkage Method for Hierarchical Clustering," *MetricGate*. [Online]. Available: <https://metricgate.com/blogs/choosing-linkage-method-hierarchical/>. [Accessed: Sep. 16, 2026].
- [9] A. Vysala and J. Gomes, "Evaluating and validating cluster results," *Computer Science & Information Technology (CS & IT)*, pp. 37–47, 2020, doi: 10.5121/csit.2020.100904.